

Geometry Calculator — ASP.NET Core Web API + HTML/JS Frontend

Step-by-Step Guide (Visual Studio 2022)

Part 1: Backend — ASP.NET Core Web API

Step 1: Create the Project

1. Open **Visual Studio 2022**
2. Click **Create a new project**
3. Select **ASP.NET Core Web API** → Click **Next**
4. Project name: **GeometryCalculator** → Choose location → Click **Next**
5. Configure:
 - Framework: **.NET 8.0**
 - Authentication: **None**
 - ☒ Configure for HTTPS
 - ☒ Use controllers
 - ☒ Enable OpenAPI support (Swagger)
6. Click **Create**

Step 2: Clean Up the Template

Delete the following auto-generated files (we won't need them):

- **WeatherForecast.cs**
- **Controllers/WeatherForecastController.cs**

Step 3: Create the Models

Right-click the project → **Add** → **New Folder** → Name it **Models**.

3a. Create **ShapeRequest.cs**

Right-click **Models** → **Add** → **Class** → Name: **ShapeRequest.cs**

```
namespace GeometryCalculator.Models
{
    public class ShapeRequest
    {
        public string Shape { get; set; } = string.Empty;
        public Dictionary<string, double> Properties { get; set; } = new();
    }
}
```

3b. Create **ShapeResult.cs**

Right-click **Models** → **Add** → **Class** → Name: **ShapeResult.cs**

```
namespace GeometryCalculator.Models
{
    public class ShapeResult
    {
        public string Shape { get; set; } = string.Empty;
        public double Area { get; set; }
        public double Circumference { get; set; }
        public Dictionary<string, double> InputProperties { get; set; } = new();
    }
}
```

Step 4: Create the Calculation Service

Right-click the project → **Add** → **New Folder** → Name it **Services**.

4a. Create **IGeometryService.cs**

Right-click **Services** → **Add** → **New Item** → **Interface** → Name: **IGeometryService.cs**

```
using GeometryCalculator.Models;

namespace GeometryCalculator.Services
{
    public interface IGeometryService
    {
        ShapeResult Calculate(ShapeRequest request);
        List<string> GetSupportedShapes();
    }
}
```

4b. Create **GeometryService.cs**

Right-click **Services** → **Add** → **Class** → Name: **GeometryService.cs**

```
using GeometryCalculator.Models;

namespace GeometryCalculator.Services
{
    public class GeometryService : IGeometryService
    {
        public List<string> GetSupportedShapes()
        {
            return new List<string>

```

```
{
    "Circle",
    "Rectangle",
    "Triangle",
    "Square",
    "Ellipse",
    "Parallelogram",
    "Trapezoid"
};
}

public ShapeResult Calculate(ShapeRequest request)
{
    var result = new ShapeResult
    {
        Shape = request.Shape,
        InputProperties = request.Properties
    };

    switch (request.Shape.ToLower())
    {
        case "circle":
            CalculateCircle(request.Properties, result);
            break;
        case "rectangle":
            CalculateRectangle(request.Properties, result);
            break;
        case "triangle":
            CalculateTriangle(request.Properties, result);
            break;
        case "square":
            CalculateSquare(request.Properties, result);
            break;
        case "ellipse":
            CalculateEllipse(request.Properties, result);
            break;
        case "parallelogram":
            CalculateParallelogram(request.Properties, result);
            break;
        case "trapezoid":
            CalculateTrapezoid(request.Properties, result);
            break;
        default:
            throw new ArgumentException($"Unsupported shape:
{request.Shape}");
    }

    // Round to 4 decimal places
    result.Area = Math.Round(result.Area, 4);
    result.Circumference = Math.Round(result.Circumference, 4);

    return result;
}
```

```
private void CalculateCircle(Dictionary<string, double> props, ShapeResult
result)
{
    double radius = GetProperty(props, "radius");
    result.Area = Math.PI * radius * radius;
    result.Circumference = 2 * Math.PI * radius;
}

private void CalculateRectangle(Dictionary<string, double> props,
ShapeResult result)
{
    double length = GetProperty(props, "length");
    double width = GetProperty(props, "width");
    result.Area = length * width;
    result.Circumference = 2 * (length + width);
}

private void CalculateTriangle(Dictionary<string, double> props,
ShapeResult result)
{
    double sideA = GetProperty(props, "sideA");
    double sideB = GetProperty(props, "sideB");
    double sideC = GetProperty(props, "sideC");

    // Heron's formula for area
    double s = (sideA + sideB + sideC) / 2;
    result.Area = Math.Sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
    result.Circumference = sideA + sideB + sideC;
}

private void CalculateSquare(Dictionary<string, double> props, ShapeResult
result)
{
    double side = GetProperty(props, "side");
    result.Area = side * side;
    result.Circumference = 4 * side;
}

private void CalculateEllipse(Dictionary<string, double> props,
ShapeResult result)
{
    double semiMajor = GetProperty(props, "semiMajorAxis");
    double semiMinor = GetProperty(props, "semiMinorAxis");
    result.Area = Math.PI * semiMajor * semiMinor;
    // Ramanujan's approximation for circumference
    double h = Math.Pow(semiMajor - semiMinor, 2) /
        Math.Pow(semiMajor + semiMinor, 2);
    result.Circumference = Math.PI * (semiMajor + semiMinor) *
        (1 + (3 * h) / (10 + Math.Sqrt(4 - 3 * h)));
}

private void CalculateParallelogram(Dictionary<string, double> props,
ShapeResult result)
{

```

```

        double baseLength = GetProperty(props, "base");
        double height = GetProperty(props, "height");
        double side = GetProperty(props, "side");
        result.Area = baseLength * height;
        result.Circumference = 2 * (baseLength + side);
    }

    private void CalculateTrapezoid(Dictionary<string, double> props,
    ShapeResult result)
    {
        double topBase = GetProperty(props, "topBase");
        double bottomBase = GetProperty(props, "bottomBase");
        double height = GetProperty(props, "height");
        double leftSide = GetProperty(props, "leftSide");
        double rightSide = GetProperty(props, "rightSide");
        result.Area = 0.5 * (topBase + bottomBase) * height;
        result.Circumference = topBase + bottomBase + leftSide + rightSide;
    }

    private double GetProperty(Dictionary<string, double> props, string key)
    {
        if (!props.ContainsKey(key))
            throw new ArgumentException($"Missing required property: {key}");

        if (props[key] <= 0)
            throw new ArgumentException($"Property '{key}' must be greater
than zero.");

        return props[key];
    }
}

```

Step 5: Create the API Controller

Right-click **Controllers** → **Add** → **Controller** → **API Controller - Empty** → Name:
GeometryController.cs

```

using GeometryCalculator.Models;
using GeometryCalculator.Services;
using Microsoft.AspNetCore.Mvc;

namespace GeometryCalculator.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class GeometryController : ControllerBase
    {
        private readonly IGeometryService _geometryService;

        public GeometryController(IGeometryService geometryService)

```

```

    {
        _geometryService = geometryService;
    }

    /// <summary>
    /// Returns the list of supported shapes.
    /// </summary>
    [HttpGet("shapes")]
    public ActionResult<List<string>> GetShapes()
    {
        return Ok(_geometryService.GetSupportedShapes());
    }

    /// <summary>
    /// Returns the required properties for a given shape.
    /// </summary>
    [HttpGet("shapes/{shapeName}/properties")]
    public ActionResult<List<string>> GetShapeProperties(string shapeName)
    {
        var propertiesMap = new Dictionary<string, List<string>>(
            StringComparer.OrdinalIgnoreCase)
        {
            { "circle",          new List<string> { "radius" } },
            { "rectangle",       new List<string> { "length", "width" } },
            { "triangle",        new List<string> { "sideA", "sideB", "sideC" } },
            { "square",          new List<string> { "side" } },
            { "ellipse",         new List<string> { "semiMajorAxis",
"semiMinorAxis" } },
            { "parallelogram",    new List<string> { "base", "height", "side" } },
            { "trapezoid",        new List<string> { "topBase", "bottomBase",
"height", "leftSide",
"rightSide" } }
        };

        if (!propertiesMap.ContainsKey(shapeName))
            return NotFound($"Shape '{shapeName}' is not supported.");

        return Ok(propertiesMap[shapeName]);
    }

    /// <summary>
    /// Calculates the area and circumference of the given shape.
    /// </summary>
    [HttpPost("calculate")]
    public ActionResult<ShapeResult> Calculate([FromBody] ShapeRequest
request)
    {
        try
        {
            var result = _geometryService.Calculate(request);
            return Ok(result);
        }
    }

```

```
        catch (ArgumentException ex)
        {
            return BadRequest(new { error = ex.Message });
        }
    }
}
```

Step 6: Register the Service & Enable Static Files

Open **Program.cs** and replace its entire content with:

```
using GeometryCalculator.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Register the Geometry Service with Dependency Injection
builder.Services.AddScoped<IGeometryService, GeometryService>();

// Add CORS policy (allows the frontend to call the API)
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAll", policy =>
    {
        policy.AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader();
    });
});

var app = builder.Build();

// Configure the HTTP request pipeline
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseCors("AllowAll");

// Enable serving static files (for the frontend)
app.UseDefaultFiles();
app.UseStaticFiles();
```

```
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Step 7: Build and Test the API

1. Press **Ctrl + Shift + B** to build
2. Press **F5** to run
3. Swagger UI will open — test the three endpoints:
 - **GET /api/geometry/shapes** → returns the list of shapes
 - **GET /api/geometry/shapes/circle/properties** → returns `["radius"]`
 - **POST /api/geometry/calculate** → with body:

```
{
  "shape": "circle",
  "properties": { "radius": 5 }
}
```

Part 2: Frontend — HTML/CSS/JavaScript

Step 8: Create the Frontend File

1. In Solution Explorer, right-click the project → **Add** → **New Folder** → Name it **wwwroot**
2. Right-click **wwwroot** → **Add** → **New Item** → **HTML Page** → Name: **index.html**

Paste the following complete frontend code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Geometry Calculator</title>
  <style>
    /* ===== Reset & Base ===== */
    *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: linear-gradient(135deg, #0f0c29, #302b63, #24243e);
      min-height: 100vh;
      display: flex;
      justify-content: center;
      align-items: flex-start;
      padding: 40px 20px;
      color: #333;
    }
  </style>
</head>
<body>
```

```
}

/* ===== Card Container ===== */
.calculator-card {
  background: #fff;
  border-radius: 16px;
  box-shadow: 0 20px 60px rgba(0, 0, 0, 0.3);
  width: 100%;
  max-width: 520px;
  overflow: hidden;
}

.card-header {
  background: linear-gradient(135deg, #667eea, #764ba2);
  color: #fff;
  padding: 28px 32px;
  text-align: center;
}

.card-header h1 {
  font-size: 1.6rem;
  font-weight: 600;
  margin-bottom: 4px;
}

.card-header p {
  font-size: 0.9rem;
  opacity: 0.85;
}

.card-body { padding: 32px; }

/* ===== Form Elements ===== */
.form-group { margin-bottom: 20px; }

.form-group label {
  display: block;
  font-weight: 600;
  margin-bottom: 8px;
  font-size: 0.9rem;
  color: #444;
}

select, input[type="number"] {
  width: 100%;
  padding: 12px 16px;
  border: 2px solid #e0e0e0;
  border-radius: 8px;
  font-size: 1rem;
  transition: border-color 0.2s;
  outline: none;
}

select:focus, input[type="number"]:focus {
```

```
        border-color: #667eea;
    }

    input[type="number"]::placeholder { color: #aaa; }

    /* ===== Properties Container ===== */
    #propertiesContainer {
        transition: all 0.3s ease;
    }

    .property-input {
        display: flex;
        align-items: center;
        gap: 12px;
        margin-bottom: 12px;
    }

    .property-input label {
        min-width: 130px;
        font-weight: 500;
        font-size: 0.88rem;
        color: #555;
        text-transform: capitalize;
    }

    .property-input input { flex: 1; }

    /* ===== Button ===== */
    .btn-calculate {
        width: 100%;
        padding: 14px;
        background: linear-gradient(135deg, #667eea, #764ba2);
        color: #fff;
        border: none;
        border-radius: 8px;
        font-size: 1.05rem;
        font-weight: 600;
        cursor: pointer;
        transition: transform 0.15s, box-shadow 0.15s;
        margin-top: 8px;
    }

    .btn-calculate:hover {
        transform: translateY(-2px);
        box-shadow: 0 6px 20px rgba(102, 126, 234, 0.4);
    }

    .btn-calculate:active { transform: translateY(0); }

    .btn-calculate:disabled {
        opacity: 0.5;
        cursor: not-allowed;
        transform: none;
        box-shadow: none;
    }
```

```
}

/* ===== Results Panel ===== */
.results-panel {
  margin-top: 24px;
  background: #f8f9ff;
  border-radius: 12px;
  padding: 24px;
  display: none;
  animation: fadeIn 0.4s ease;
}

.results-panel.visible { display: block; }

.results-panel h3 {
  font-size: 1rem;
  color: #667eea;
  margin-bottom: 16px;
  text-align: center;
}

.result-row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 12px 0;
  border-bottom: 1px solid #e8eaf6;
}

.result-row:last-child { border-bottom: none; }

.result-label {
  font-weight: 500;
  color: #555;
}

.result-value {
  font-size: 1.2rem;
  font-weight: 700;
  color: #302b63;
}

/* ===== Error Message ===== */
.error-message {
  background: #fff0f0;
  color: #d32f2f;
  padding: 12px 16px;
  border-radius: 8px;
  margin-top: 16px;
  font-size: 0.9rem;
  display: none;
  animation: fadeIn 0.3s ease;
}
```

```

.error-message.visible { display: block; }

/* ===== Loading Spinner ===== */
.spinner {
  display: inline-block;
  width: 18px;
  height: 18px;
  border: 3px solid rgba(255,255,255,0.3);
  border-radius: 50%;
  border-top-color: #fff;
  animation: spin 0.7s linear infinite;
  vertical-align: middle;
  margin-right: 8px;
}

@keyframes spin { to { transform: rotate(360deg); } }
@keyframes fadeIn { from { opacity: 0; transform: translateY(8px); } to {
opacity: 1; transform: translateY(0); } }

/* ===== Responsive ===== */
@media (max-width: 480px) {
  .property-input { flex-direction: column; align-items: flex-start;
gap: 4px; }
  .property-input label { min-width: auto; }
}
</style>
</head>
<body>

<div class="calculator-card">
  <!-- Header -->
  <div class="card-header">
    <h1><img alt="Calculator icon" data-bbox="260 593 278 608"/> Geometry Calculator</h1>
    <p>Calculate area & circumference of shapes</p>
  </div>

  <div class="card-body">
    <!-- Step 1: Select Shape -->
    <div class="form-group">
      <label for="shapeSelect">1. Select the Shape</label>
      <select id="shapeSelect">
        <option value="">-- Choose a shape --</option>
      </select>
    </div>

    <!-- Step 2: Enter Properties (dynamic) -->
    <div class="form-group" id="propertiesSection" style="display: none;">
      <label>2. Enter the Shape Properties</label>
      <div id="propertiesContainer"></div>
    </div>

    <!-- Step 3: Calculate -->
    <button class="btn-calculate" id="btnCalculate" disabled>
      Calculate

```

```

    </button>

    <!-- Error Message -->
    <div class="error-message" id="errorMessage"></div>

    <!-- Results -->
    <div class="results-panel" id="resultsPanel">
        <h3 id="resultsTitle">Results</h3>
        <div class="result-row">
            <span class="result-label">Area</span>
            <span class="result-value" id="resultArea">—</span>
        </div>
        <div class="result-row">
            <span class="result-label">Circumference / Perimeter</span>
            <span class="result-value" id="resultCircumference">—</span>
        </div>
    </div>
</div>
</div>

<script>
    // ===== Configuration =====
    const API_BASE = '/api/geometry';

    // ===== DOM References =====
    const shapeSelect = document.getElementById('shapeSelect');
    const propertiesSection = document.getElementById('propertiesSection');
    const propertiesContainer =
document.getElementById('propertiesContainer');
    const btnCalculate = document.getElementById('btnCalculate');
    const resultsPanel = document.getElementById('resultsPanel');
    const resultsTitle = document.getElementById('resultsTitle');
    const resultArea = document.getElementById('resultArea');
    const resultCircumference =
document.getElementById('resultCircumference');
    const errorMessage = document.getElementById('errorMessage');

    // ===== Helper: Friendly Labels =====
    function friendlyLabel(key) {
        const labels = {
            'radius': 'Radius',
            'length': 'Length',
            'width': 'Width',
            'side': 'Side',
            'sideA': 'Side A',
            'sideB': 'Side B',
            'sideC': 'Side C',
            'semiMajorAxis': 'Semi-Major Axis',
            'semiMinorAxis': 'Semi-Minor Axis',
            'base': 'Base',
            'height': 'Height',
            'topBase': 'Top Base',
            'bottomBase': 'Bottom Base',
            'leftSide': 'Left Side',

```

```
        'rightSide': 'Right Side'
    };
    return labels[key] || key;
}

// ===== Helper: Show / Hide Error =====
function showError(msg) {
    errorMessage.textContent = msg;
    errorMessage.classList.add('visible');
    resultsPanel.classList.remove('visible');
}

function hideError() {
    errorMessage.classList.remove('visible');
}

// ===== 1. Load Shapes on Page Load =====
async function loadShapes() {
    try {
        const response = await fetch(`${API_BASE}/shapes`);
        const shapes = await response.json();

        shapes.forEach(shape => {
            const option = document.createElement('option');
            option.value = shape;
            option.textContent = shape;
            shapeSelect.appendChild(option);
        });
    } catch (err) {
        showError('Failed to load shapes. Is the API running?');
    }
}

// ===== 2. On Shape Selection – Load Properties =====
shapeSelect.addEventListener('change', async () => {
    const shape = shapeSelect.value;
    propertiesContainer.innerHTML = '';
    resultsPanel.classList.remove('visible');
    hideError();

    if (!shape) {
        propertiesSection.style.display = 'none';
        btnCalculate.disabled = true;
        return;
    }

    try {
        const response = await fetch(
            `${API_BASE}/shapes/${shape}/properties`
        );
        const properties = await response.json();

        properties.forEach(prop => {
            const div = document.createElement('div');
```

```

        div.className = 'property-input';

        const label = document.createElement('label');
        label.textContent = friendlyLabel(prop);
        label.setAttribute('for', `prop_${prop}`);

        const input = document.createElement('input');
        input.type = 'number';
        input.id = `prop_${prop}`;
        input.name = prop;
        input.placeholder = `Enter
${friendlyLabel(prop).toLowerCase()}`;
        input.min = '0.01';
        input.step = 'any';

        div.appendChild(label);
        div.appendChild(input);
        propertiesContainer.appendChild(div);
    });

    propertiesSection.style.display = 'block';
    btnCalculate.disabled = false;
} catch (err) {
    showError('Failed to load shape properties.');
```

```

// ===== 3. On Calculate – POST to API =====
```

```

btnCalculate.addEventListener('click', async () => {
    hideError();
    resultsPanel.classList.remove('visible');

    const shape = shapeSelect.value;
    if (!shape) return;

    // Gather property values
    const inputs = propertiesContainer.querySelectorAll('input');
    const properties = {};
    let valid = true;

    inputs.forEach(input => {
        const val = parseFloat(input.value);
        if (isNaN(val) || val <= 0) {
            valid = false;
            input.style.borderColor = '#d32f2f';
        } else {
            input.style.borderColor = '#e0e0e0';
            properties[input.name] = val;
        }
    });

    if (!valid) {
        showError('Please enter valid positive numbers for all
properties.');
```

```
        return;
    }

    // Show loading state
    btnCalculate.disabled = true;
    btnCalculate.innerHTML =
        '<span class="spinner"></span> Calculating...';

    try {
        const response = await fetch(`${API_BASE}/calculate`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ shape, properties })
        });

        if (!response.ok) {
            const err = await response.json();
            throw new Error(err.error || 'Calculation failed.');
        }

        const result = await response.json();

        // Display results
        resultsTitle.textContent = `Results for ${result.shape}`;
        resultArea.textContent = result.area.toLocaleString(
            undefined, { maximumFractionDigits: 4 }
        ) + ' sq. units';
        resultCircumference.textContent =
            result.circumference.toLocaleString(
                undefined, { maximumFractionDigits: 4 }
            ) + ' units';

        resultsPanel.classList.add('visible');
    } catch (err) {
        showError(err.message);
    } finally {
        btnCalculate.disabled = false;
        btnCalculate.textContent = 'Calculate';
    }
});

// ===== Allow Enter key to trigger calculation =====
document.addEventListener('keydown', (e) => {
    if (e.key === 'Enter' && !btnCalculate.disabled) {
        btnCalculate.click();
    }
});

// ===== Initialise =====
loadShapes();
</script>
```

</body>

</html>

Part 3: Run the Complete Application

Step 9: Launch and Verify

1. Press **F5** in Visual Studio to run the project
2. The browser will open to the Swagger page. Change the URL to the **root** of the site:

https://localhost:{port}/

- (Remove `/swagger/index.html` from the URL)
3. You should see the Geometry Calculator frontend

Step 10: Test the Workflow

1. **Select a shape** from the dropdown (e.g., Circle)
2. The relevant **property inputs** appear dynamically (e.g., Radius)
3. **Enter values** (e.g., Radius = 5)
4. Click **Calculate**
5. The **Area** (78.5398 sq. units) and **Circumference** (31.4159 units) appear below

Project Structure (Final)

GeometryCalculator/

Controllers/

GeometryController.cs

Models/

ShapeRequest.cs

ShapeResult.cs

Services/

IGeometryService.cs

GeometryService.cs

wwwroot/

index.html

Program.cs

GeometryCalculator.csproj

← API endpoints

← Input model

← Output model

← Service interface

← Calculation logic

← Frontend (HTML/CSS/JS)

← App configuration

API Endpoints Summary

Method	Endpoint	Description
GET	/api/geometry/shapes	Returns list of supported shapes
GET	/api/geometry/shapes/{name}/properties	Returns required properties for a shape

17 / 19

Method	Endpoint	Description
POST	/api/geometry/calculate	Calculates area & circumference

Sample POST Request Body

```
{
  "shape": "rectangle",
  "properties": {
    "length": 10,
    "width": 5
  }
}
```

Sample Response

```
{
  "shape": "Rectangle",
  "area": 50.0,
  "circumference": 30.0,
  "inputProperties": {
    "length": 10.0,
    "width": 5.0
  }
}
```

Key Design Decisions Explained

Why a Service layer? Separating calculation logic into `GeometryService` follows the Single Responsibility Principle. The controller handles HTTP concerns; the service handles geometry. This makes unit testing the calculations straightforward without needing to spin up an HTTP pipeline.

Why `Dictionary<string, double>` for properties? This avoids creating a separate request model for every shape. The API dynamically tells the frontend what properties each shape needs, and the frontend dynamically builds the form. Adding a new shape requires updating only the service and the properties map — no frontend changes needed.

Why static files in `wwwroot`? Keeps the entire solution in one project for simplicity. For production, you'd likely separate the frontend into its own SPA project (React, Angular, Blazor) and communicate via the API.

Optional Enhancements

To extend this project, consider adding: unit tests with xUnit for the `GeometryService`, input validation using FluentValidation or Data Annotations, a shape diagram (SVG) rendered dynamically on the frontend,

additional shapes such as Pentagon or Hexagon, a calculation history panel using `localStorage`, or deploying to Azure App Service with a CI/CD pipeline from GitHub Actions.